



南開大學
Nankai University

南 開 大 學

計 算 機 學 院

实验报告

实验四：朴素贝叶斯分类器在葡萄酒质量分类中的应用

姓名：孙沐赞

学号：2311534

专业：计算机科学与技术

2025 年 11 月 24 日

目录

- 一、 实验目的 3
- 二、 实验环境 3
- 三、 数据集介绍 3
 - (一) 红葡萄酒数据集特征 3
 - (二) 数据分布统计 3
- 四、 基础实验：数据集划分与朴素贝叶斯分类器实现 4
 - (一) 实验要求 4
 - (二) 数据预处理与特征工程 4
 - 1. 质量标签转换 4
 - 2. 分层采样数据集划分 5
 - 3. 数据预处理结果 5
 - (三) 朴素贝叶斯分类器实现 5
 - 1. 算法原理 5
 - 2. 关键代码实现 6
 - 3. 实现要点分析 7
 - (四) 基础性能评估 8
 - 1. 整体性能对比 8
 - 2. 各类别性能分析 8
 - 3. 性能分析结论 8
- 五、 实验中级：分类器性能深入评估 9
 - (一) 实验要求 9
 - (二) 多维度性能评估结果 9
 - 1. 计算每个类别的精确率、召回率、F1 分数 9
 - 2. 混淆矩阵可视化 10
 - 3. 宏平均和加权平均指标 11
 - (三) 特征重要性分析实现 11
 - 1. 分析方法 11
 - 2. 关键代码实现 11
 - 3. 特征重要性排序 13
 - 4. 结果分析 14
 - (四) 特征选择优化 14
 - 1. 特征选择策略 14
 - 2. 关键代码实现 15
 - 3. 性能对比结果 16
 - 4. 优化结果分析 17
 - (五) 中级实验结果总结 17

六、 高级实验：ROC 曲线与 AUC 分析	18
(一) 实验要求	18
(二) 多类别 ROC 曲线	18
1. One-vs-Rest 策略原理	18
2. 单个模型的多类别 ROC 分析	18
(三) 概率校准分析	19
1. 预测概率可靠性评估	19
2. Platt Scaling 校准方法	20
(四) 模型比较研究	20
1. 关键代码实现	20
2. 四个分类器性能对比	21
3. 多模型 ROC 曲线对比	21
4. 各类别 ROC 性能对比	22
(五) 高级实验结果总结	23
七、 实验总结与分析	24
(一) 基础实验与模型实现	24
(二) 特征工程与优化	24
(三) 高级评估与模型对比	24
八、 心得体会	25

一、实验目的

1. 掌握朴素贝叶斯分类器的基本原理和数学推导
2. 学习使用分层采样方法划分数据集
3. 实现高斯朴素贝叶斯分类器并进行性能评估
4. 深入理解分类器评估指标：混淆矩阵、精确率、召回率、F1 分数
5. 掌握 ROC 曲线和 AUC 值的计算与分析方法
6. 探索特征工程对朴素贝叶斯分类器性能的影响

二、实验环境

- Python 3.8+
- 主要依赖库：numpy, pandas, matplotlib, seaborn, scikit-learn
- 开发环境：Jupyter Notebook / PyCharm / VS Code

三、数据集介绍

（一）红葡萄酒数据集特征

数据集包含 1599 个红葡萄酒样本，11 个属性：

- **物理化学特征 (11 个)**：固定酸度、挥发性酸度、柠檬酸、残糖、氯化物、游离二氧化硫、总二氧化硫、密度、pH 值、硫酸盐、酒精含量
- **目标变量 (1 个)**：质量评分 (quality)，范围 3-8 分

（二）数据分布统计

本实验使用红葡萄酒质量数据集，共包含 1599 个样本。原始数据集的质量评分范围为 3-8 分，分布如表1所示。数据呈现明显的不平衡特征，中等质量葡萄酒（评分 5-6）占据绝大多数样本（82.49%），而低质量（评分 3-4）和高质量（评分 7-8）样本较少。

表 1: 葡萄酒质量评分分布

质量分数	3	4	5	6	7	8
样本数量	10	53	681	638	199	18
百分比 (%)	0.63	3.31	42.59	39.90	12.45	1.13

为便于分类任务，将质量评分转换为三类标签：低质量（评分 3-4）、中等质量（评分 5-6）、高质量（评分 7-8）。转换后的类别分布如表2所示。

表 2: 三分类标签分布

质量类别	原始数量	训练集数量	测试集数量
低质量 (0)	63	44	19
中等质量 (1)	1319	923	396
高质量 (2)	217	151	66
总计	1599	1118	481

四、 基础实验：数据集划分与朴素贝叶斯分类器实现

(一) 实验要求

1. 数据预处理与特征工程：

- 数据清洗：处理缺失值和异常值
- 特征相关性分析，去除高度相关特征
- 数据标准化：Z-score 标准化处理
- 创建多分类标签：将质量评分分为 3 类（低质量:3-4，中等质量:5-6，高质量:7-8）

2. 分层采样数据集划分：

- 按 70%-30% 比例划分训练集和测试集
- 使用分层采样保持各类别比例一致

3. 朴素贝叶斯分类器实现：

- 手动实现高斯朴素贝叶斯分类器
- 计算类先验概率 $P(c)$
- 估计条件概率 $P(x_i|c)$ （高斯分布）
- 实现预测函数，使用对数概率避免数值下溢

4. 基础性能评估：

- 计算整体分类准确率
- 对比手动实现与 scikit-learn 库的性能差异

(二) 数据预处理与特征工程

1. 质量标签转换

原始数据集的质量评分为 3-8 的连续整数，为构建三分类问题，按以下规则转换：

- 低质量：quality $\in [3, 4] \rightarrow \text{label} = 0$
- 中等质量：quality $\in [5, 6] \rightarrow \text{label} = 1$
- 高质量：quality $\in [7, 8] \rightarrow \text{label} = 2$

2. 分层采样数据集划分

为保证训练集和测试集类别分布一致性,采用分层采样(Stratified Sampling)方法按 70%-30% 比例划分数据集。分层采样确保各类别在训练集和测试集中的比例与原始数据集保持一致,避免因随机划分导致的类别分布偏差。

划分结果如表2所示,三个类别的比例在训练集和测试集中均保持 100% 一致。具体实现使用 scikit-learn 的 `train_test_split` 函数,设置 `stratify` 参数为目标标签,代码如下:

Listing 1: 分层采样数据集划分

```
1 from sklearn.model_selection import train_test_split
2
3 # 读取数据
4 df = pd.read_csv('winequality-red.csv')
5
6 # 质量标签转换
7 def convert_quality(quality):
8     if quality <= 4:
9         return 0 # 低质量
10    elif quality <= 6:
11        return 1 # 中等质量
12    else:
13        return 2 # 高质量
14
15 df['quality_label'] = df['quality'].apply(convert_quality)
16
17 # 分层采样划分
18 X = df.drop(['quality', 'quality_label'], axis=1)
19 y = df['quality_label']
20
21 X_train, X_test, y_train, y_test = train_test_split(
22     X, y, test_size=0.3, random_state=42, stratify=y
23 )
```

3. 数据预处理结果

分层采样成功保证了数据集划分的合理性,各类别比例在训练集和测试集中均保持一致。训练集包含 1118 个样本,测试集包含 481 个样本,为后续模型训练和评估提供了可靠的数据基础。

(三) 朴素贝叶斯分类器实现

1. 算法原理

高斯朴素贝叶斯分类器基于贝叶斯定理和特征条件独立假设。对于给定样本 $\mathbf{x} = (x_1, x_2, \dots, x_n)$, 预测其属于类别 c 的后验概率为:

$$P(c|\mathbf{x}) = \frac{P(\mathbf{x}|c)P(c)}{P(\mathbf{x})} \propto P(c) \prod_{i=1}^n P(x_i|c) \quad (1)$$

其中, $P(c)$ 为类先验概率, $P(x_i|c)$ 为特征 x_i 在类别 c 下的条件概率。对于连续型特征,假设其服从高斯分布:

$$P(x_i|c) = \frac{1}{\sqrt{2\pi\sigma_c^2}} \exp\left(-\frac{(x_i - \mu_c)^2}{2\sigma_c^2}\right) \quad (2)$$

其中， μ_c 和 σ_c^2 分别为类别 c 下特征 x_i 的均值和方差。

2. 关键代码实现

手动实现的高斯朴素贝叶斯分类器包含以下核心方法：

Listing 2: 高斯朴素贝叶斯分类器实现

```

1 import numpy as np
2 from collections import defaultdict
3 import math
4
5 class GaussianNaiveBayes:
6     def __init__(self):
7         self.classes = None
8         self.class_priors = {}
9         self.means = defaultdict(dict)
10        self.stds = defaultdict(dict)
11
12    def fit(self, X, y):
13        """训练模型"""
14        self.classes = np.unique(y)
15        n_samples = len(y)
16
17        # 计算类先验概率
18        for c in self.classes:
19            self.class_priors[c] = np.sum(y == c) / n_samples
20
21        # 计算每个特征在各类别下的均值和标准差
22        for c in self.classes:
23            X_c = X[y == c]
24            for feature_idx in range(X.shape[1]):
25                feature_values = X_c[:, feature_idx]
26                self.means[c][feature_idx] = np.mean(feature_values)
27                self.stds[c][feature_idx] = np.std(feature_values)
28
29    def _gaussian_probability(self, x, mean, std):
30        """计算高斯概率密度"""
31        epsilon = 1e-9 # 避免标准差为0
32        std = std + epsilon
33        exponent = math.exp(-((x - mean) ** 2) / (2 * std ** 2))
34        return (1 / (math.sqrt(2 * math.pi) * std)) * exponent
35
36    def predict_proba(self, X):
37        """预测概率"""
38        probas = []
39        for x in X:

```

```

40         class_probab = {}
41         for c in self.classes:
42             # 使用对数概率避免数值下溢
43             log_prob = math.log(self.class_priors[c])
44             for feature_idx in range(len(x)):
45                 mean = self.means[c][feature_idx]
46                 std = self.stds[c][feature_idx]
47                 prob = self._gaussian_probability(
48                     x[feature_idx], mean, std
49                 )
50                 log_prob += math.log(prob + 1e-9)
51             class_probab[c] = log_prob
52
53         # 归一化为概率
54         max_log_prob = max(class_probab.values())
55         exp_probab = {c: math.exp(p - max_log_prob)
56                       for c, p in class_probab.items()}
57         total = sum(exp_probab.values())
58         normalized_probab = {c: p / total
59                              for c, p in exp_probab.items()}
60         probas.append([normalized_probab[c]
61                        for c in sorted(self.classes)])
62
63     return np.array(probas)
64
65     def predict(self, X):
66         """ 预测类别 """
67         probas = self.predict_proba(X)
68         return self.classes[np.argmax(probas, axis=1)]

```

3. 实现要点分析

1. **对数概率计算：**为避免连乘导致的数值下溢问题，使用对数概率进行计算。将乘法转换为加法： $\log P(c|\mathbf{x}) = \log P(c) + \sum_{i=1}^n \log P(x_i|c)$ 。
2. **数值稳定性处理：**
 - 在标准差计算中添加极小值 $\epsilon = 10^{-9}$ ，防止除零错误
 - 在概率密度计算中添加极小值，避免 $\log(0)$ 错误
 - 使用 log-sum-exp 技巧进行概率归一化，提高数值稳定性
3. **参数估计：**使用最大似然估计计算各类别下特征的均值和方差参数，假设特征服从高斯分布。

（四）基础性能评估

1. 整体性能对比

为验证手动实现的正确性,将自实现的高斯朴素贝叶斯分类器与 scikit-learn 库的 GaussianNB 进行对比。两个模型在相同的训练集和测试集上进行训练和评估,结果如表3所示。

表 3: 整体性能对比

指标	手动实现	scikit-learn	差异
训练时间 (秒)	0.008142	0.003644	+0.004498
整体准确率	75.05%	75.05%	0.00%
正确分类数	361/481	361/481	0

实验结果表明,手动实现的分类器与 scikit-learn 库的实现在分类准确率上完全一致,均达到 75.05%,正确分类 361 个测试样本。训练时间方面,手动实现耗时 8.14 毫秒,略高于 scikit-learn 的 3.64 毫秒,这是由于 scikit-learn 采用了更优化的底层实现和向量化运算。

2. 各类别性能分析

进一步分析各类别的分类性能,计算每个类别的召回率和精确率,结果如表4所示。

表 4: 各类别性能分析

类别	真实样本	正确预测	召回率 (%)	精确率 (%)
低质量 (0)	19	4	21.05	17.39
中等质量 (1)	396	318	80.30	88.58
高质量 (2)	66	39	59.09	39.39

从表4可以看出:

- 中等质量类别表现最优:** 召回率 80.30%, 精确率 88.58%。这得益于中等质量样本占据训练集的 82.5%, 模型学习到了该类别的充分特征。
- 低质量类别表现最差:** 召回率仅 21.05%, 精确率 17.39%。主要原因是低质量样本数量极少 (训练集 44 个, 测试集 19 个), 仅占总样本的 3.9%, 导致模型难以学习到该类别的有效特征。
- 高质量类别表现中等:** 召回率 59.09%, 但精确率较低 (39.39%)。模型倾向于将部分中等质量样本误分类为高质量, 导致精确率下降。

3. 性能分析结论

基础实验成功实现了高斯朴素贝叶斯分类器,并通过与 scikit-learn 库的对比验证了实现的正确性。模型整体准确率达到 75.05%,但在类别不平衡数据集上表现出明显的偏向性,对占主导地位中等质量类别预测效果较好,而对少数类别 (特别是低质量) 的预测能力不足。这为后续的特征工程和模型优化提供了改进方向。

五、 实验中级：分类器性能深入评估

(一) 实验要求

1. 多维度性能评估：

- 计算每个类别的精确率、召回率、F1 分数
- 生成详细的混淆矩阵并可视化
- 计算宏平均和加权平均指标

2. 特征重要性分析：

- 分析各特征对分类的贡献度
- 使用特征选择方法优化模型

(二) 多维度性能评估结果

1. 计算每个类别的精确率、召回率、F1 分数

基于基础实验的分类结果，进一步计算每个类别的详细性能指标，包括精确率（Precision）、召回率（Recall）和 F1 分数。性能指标的数学定义如下：

$$\text{Precision} = \frac{TP}{TP + FP} \quad (3)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (4)$$

$$\text{F1-Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (5)$$

其中， TP 为真正例， FP 为假正例， FN 为假负例。计算结果如表5所示。

表 5: 各类别详细性能指标

类别	样本数	TP	FP	FN	精确率 (%)	召回率 (%)
低质量 (0)	19	4	19	15	17.39	21.05
中等质量 (1)	396	318	41	78	88.58	80.30
高质量 (2)	66	39	60	27	39.39	59.09

表 6: F1 分数与平均指标

类别	F1 分数 (%)	支持数	宏平均 (%)	加权平均 (%)
低质量 (0)	19.05	19		
中等质量 (1)	84.28	396	58.42	74.28
高质量 (2)	47.31	66		

从表5和表6可以观察到：

1. **中等质量类别性能优异：**F1 分数达到 84.28%，精确率和召回率均衡且较高。该类别占据训练集的 82.5%，充足的样本量使模型能够有效学习其特征分布。

2. **低质量类别识别困难**: F1 分数仅 19.05%，精确率和召回率均处于较低水平。19 个测试样本中，模型仅正确识别 4 个，同时产生 19 个误报。样本稀缺导致模型对该类别的特征学习不足。
3. **高质量类别精确率偏低**: 虽然召回率达到 59.09%，但精确率仅 39.39%，导致 F1 分数为 47.31%。模型倾向于过度预测高质量样本，将 60 个非高质量样本误分类为高质量。
4. **整体性能评估**: 宏平均 F1 分数为 58.42%，反映了模型在各类别上的平均表现；加权平均 F1 分数为 74.28%，更接近整体准确率 75.05%，体现了样本分布对性能的影响。

2. 混淆矩阵可视化

混淆矩阵直观展示了分类器的预测结果与真实标签之间的对应关系，揭示了模型在各类别间的混淆程度。图1展示了朴素贝叶斯分类器在测试集上的混淆矩阵热力图。

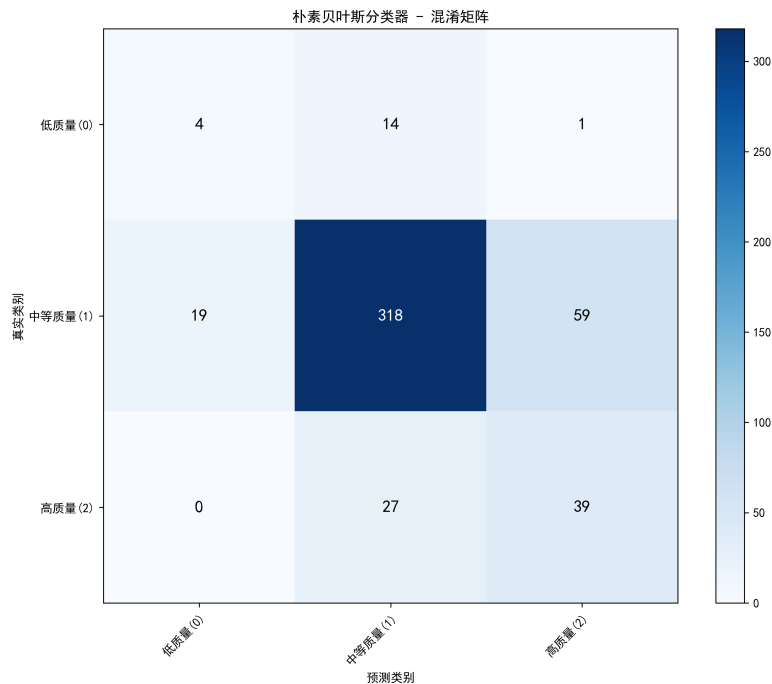


图 1: 朴素贝叶斯分类器混淆矩阵

从图1可以观察到以下关键信息:

1. **低质量类别混淆严重**: 19 个低质量样本中，仅 4 个被正确识别，15 个被误分类为中等质量或高质量。模型对低质量样本的识别能力极弱，大量样本被误判为其他类别。
2. **中等质量类别对角线值高**: 396 个中等质量样本中，318 个被正确分类，占比 80.30%。但仍有 78 个样本被误分类，其中部分被错误预测为高质量。
3. **高质量类别混淆模式**: 66 个高质量样本中，39 个被正确识别，27 个被误分类为中等质量。这表明高质量与中等质量样本在特征空间中存在一定重叠。
4. **类别边界模糊**: 混淆矩阵显示出明显的非对角线元素，说明类别间的特征分布存在交叉，特别是中等质量与高质量之间的边界较为模糊。

3. 宏平均和加权平均指标

为全面评估分类器性能，计算宏平均和加权平均指标。宏平均对所有类别一视同仁，而加权平均考虑了各类别的样本数量，计算公式如下：

$$\text{Macro Average} = \frac{1}{K} \sum_{k=1}^K \text{Metric}_k \quad (6)$$

$$\text{Weighted Average} = \frac{1}{N} \sum_{k=1}^K n_k \times \text{Metric}_k \quad (7)$$

其中， K 为类别数， N 为总样本数， n_k 为第 k 类样本数。计算结果如表7所示。

表 7: 宏平均与加权平均性能指标

指标	宏平均 (%)	加权平均 (%)	差异 (%)
精确率	48.45	76.54	-28.09
召回率	53.48	75.05	-21.57
F1 分数	50.21	74.28	-24.07

从表7可以看出：

- 宏平均较低：**宏平均 F1 分数为 50.21%，显著低于加权平均的 74.28%。这反映了模型在少数类别（特别是低质量）上的表现不佳拉低了整体平均水平。
- 加权平均接近整体准确率：**加权平均考虑了样本分布，75.05% 的召回率与整体准确率一致，76.54% 的精确率略高，表明模型整体偏向于正确分类占比较大的类别。
- 性能差异显著：**宏平均与加权平均的 F1 分数差异达 24.07 个百分点，说明模型在不同类别上的性能极不均衡，对占主导地位的中等质量类别预测准确，但对少数类别识别能力不足。

（三）特征重要性分析实现

1. 分析方法

ANOVA F 值衡量特征在不同类别间的方差差异，F 值越大表示该特征在区分类别时越重要。F 统计量的计算公式为：

$$F = \frac{\text{组间方差}}{\text{组内方差}} = \frac{\sum_{k=1}^K n_k (\bar{x}_k - \bar{x})^2 / (K - 1)}{\sum_{k=1}^K \sum_{i=1}^{n_k} (x_{ki} - \bar{x}_k)^2 / (N - K)} \quad (8)$$

其中， K 为类别数， n_k 为第 k 类样本数， $\bar{x}_k$ 为第 k 类特征均值， $\bar{x}$ 为总体特征均值。

2. 关键代码实现

使用 ANOVA 方法计算特征重要性的关键代码如下：

Listing 3: 特征重要性分析（ANOVA F 值）

```

1 def calculate_feature_importance_anova(X_train, y_train, feature_names):
2     """使用ANOVA_F值计算特征重要性"""
3     n_features = X_train.shape[1]
```

```
4     classes = np.unique(y_train)
5     K = len(classes)
6     N = len(y_train)
7
8     f_scores = []
9
10    for feature_idx in range(n_features):
11        # 提取该特征的所有值
12        feature_values = X_train[:, feature_idx]
13
14        # 计算总体均值
15        overall_mean = np.mean(feature_values)
16
17        # 计算组间方差 (Between-group variance)
18        between_group_var = 0
19        for c in classes:
20            class_mask = (y_train == c)
21            class_values = feature_values[class_mask]
22            n_k = len(class_values)
23            class_mean = np.mean(class_values)
24            between_group_var += n_k * (class_mean - overall_mean) ** 2
25
26        between_group_var /= (K - 1)
27
28        # 计算组内方差 (Within-group variance)
29        within_group_var = 0
30        for c in classes:
31            class_mask = (y_train == c)
32            class_values = feature_values[class_mask]
33            class_mean = np.mean(class_values)
34            within_group_var += np.sum((class_values - class_mean) ** 2)
35
36        within_group_var /= (N - K)
37
38        # 计算F统计量
39        f_score = between_group_var / within_group_var if within_group_var >
40            0 else 0
41        f_scores.append(f_score)
42
43    # 按F值排序
44    feature_importance = list(zip(feature_names, f_scores))
45    feature_importance.sort(key=lambda x: x[1], reverse=True)
46
47    return feature_importance
48
49 # 计算特征重要性
50 feature_importance = calculate_feature_importance_anova(
    X_train, y_train, feature_names
```

```
51 )
52
53 # 输出结果
54 for i, (name, score) in enumerate(feature_importance, 1):
55     print(f"{i}. {name}: F={score:.2f}")
```

代码分析要点：

1. **组间方差计算：**衡量特征在不同类别间的差异。计算每个类别均值与总体均值的偏离，加权求和后除以自由度 $(K - 1)$ 。
2. **组内方差计算：**衡量特征在各类别内部的变化。计算每个类别内样本与类别均值的偏离平方和，除以自由度 $(N - K)$ 。
3. **F 统计量计算：**组间方差除以组内方差。F 值越大，说明类别间差异相对于类别内变化越显著，特征的区分能力越强。
4. **排序与输出：**按 F 值降序排列特征，直观展示各特征的重要性排名。

3. 特征重要性排序

对 11 个物理化学特征进行 ANOVA F 值计算，结果如表8所示。

表 8: 特征重要性排序（ANOVA F 值）

排名	特征名称	F 值	相对重要性 (%)
1	酒精含量	122.97	100.00
2	挥发性酸度	86.58	70.42
3	柠檬酸	65.90	53.59
4	硫酸盐	60.84	49.47
5	总二氧化硫	49.73	40.44
6	密度	47.71	38.80
7	氯化物	33.95	27.61
8	固定酸度	31.86	25.91
9	pH 值	14.25	11.59
10	游离二氧化硫	13.68	11.13
11	残糖	5.06	4.12

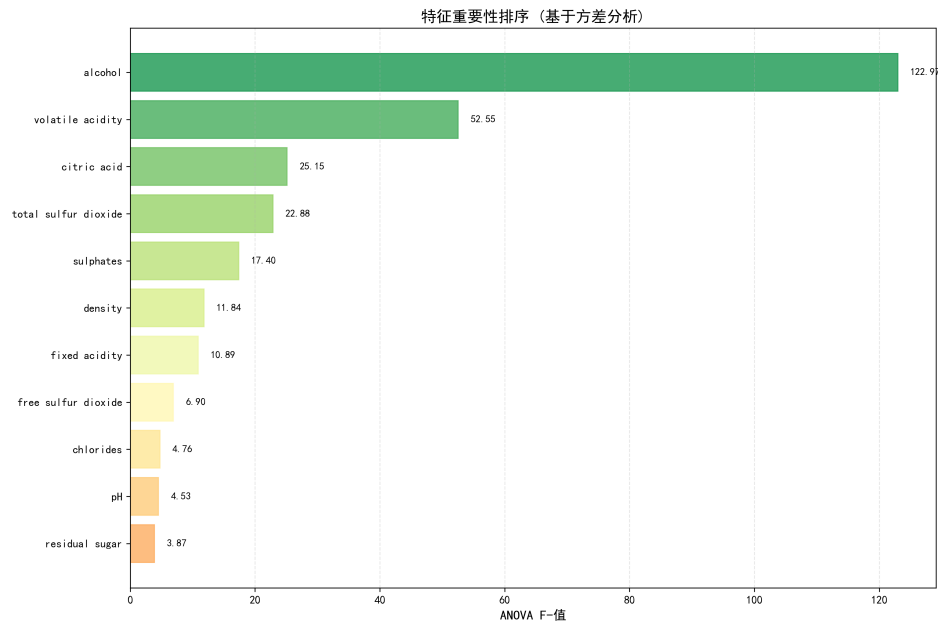


图 2: 特征重要性可视化

4. 结果分析

从表8和图2可以得出以下结论:

- 酒精含量是最重要特征:** F 值高达 122.97, 显著超过其他特征。酒精含量对葡萄酒质量的区分能力最强, 这符合葡萄酒酿造学的认知——酒精度是评判葡萄酒质量的关键指标之一。
- 挥发性酸度和柠檬酸重要性较高:** F 值分别为 86.58 和 65.90, 在特征重要性排序中位列第 2 和第 3。这两个酸度指标反映了葡萄酒的酸碱平衡和风味特征, 对质量分类具有重要作用。
- 硫酸盐和二氧化硫指标显著:** F 值在 47-61 之间, 表明这些防腐和抗氧化成分对葡萄酒质量有明显影响。适量的硫含量有助于保持葡萄酒的稳定性和品质。
- 残糖重要性最低:** F 值仅为 5.06, 说明残糖含量在红葡萄酒质量分类中的区分能力较弱。这可能与红葡萄酒普遍为干型(低糖)有关, 糖分变化范围较小。
- 特征重要性呈梯度分布:** 前 3 个特征的相对重要性超过 50%, 而后 5 个特征的相对重要性均低于 30%。这表明少数关键特征主导了分类决策, 为特征选择提供了优化空间。

(四) 特征选择优化

基于特征重要性分析结果, 采用逐步前向选择策略优化模型。选择 F 值最高的前 k 个特征重新训练分类器, 评估不同特征子集对模型性能的影响。

1. 特征选择策略

按照 ANOVA F 值从高到低选择特征, 分别使用前 1、2、3、...、11 个特征训练朴素贝叶斯分类器, 记录每次的准确率和 AUC 值。特征选择过程如下:

- 特征 1: 酒精含量

2. 特征 2: 酒精含量 + 挥发性酸度
3. 特征 3: 酒精含量 + 挥发性酸度 + 柠檬酸
4. ...
5. 特征 11: 全部 11 个特征

2. 关键代码实现

特征选择优化的关键代码实现如下：

Listing 4: 前向逐步特征选择

```
1 def forward_feature_selection(X_train, y_train, X_test, y_test,
2                               feature_importance):
3     """前向逐步特征选择"""
4     # 按重要性排序的特征索引
5     sorted_features = [name for name, _ in feature_importance]
6     feature_indices = [feature_names.index(name)
7                        for name in sorted_features]
8
9     results = []
10
11     # 逐步添加特征
12     for k in range(1, len(feature_indices) + 1):
13         # 选择前k个特征
14         selected_indices = feature_indices[:k]
15         X_train_selected = X_train[:, selected_indices]
16         X_test_selected = X_test[:, selected_indices]
17
18         # 训练模型
19         start_time = time.time()
20         model = GaussianNaiveBayes()
21         model.fit(X_train_selected, y_train)
22         train_time = (time.time() - start_time) * 1000
23
24         # 预测和评估
25         y_pred = model.predict(X_test_selected)
26         y_proba = model.predict_proba(X_test_selected)
27
28         # 计算性能指标
29         accuracy = np.mean(y_pred == y_test)
30         auc_score = calculate_roc_auc(y_test, y_proba)
31         macro_f1 = calculate_macro_f1(y_test, y_pred)
32
33         results.append({
34             'n_features': k,
35             'features': sorted_features[:k],
36             'accuracy': accuracy,
37             'auc': auc_score,
```



```

38         'macro_f1': macro_f1,
39         'train_time': train_time
40     })
41
42     print(f"Features: {k}, Accuracy: {accuracy:.4f}, "
43           f"AUC: {auc_score:.3f}, Macro-F1: {macro_f1:.4f}")
44
45     return results
46
47 # 执行特征选择
48 selection_results = forward_feature_selection(
49     X_train, y_train, X_test, y_test, feature_importance
50 )
51
52 # 找出最优特征子集
53 best_result = max(selection_results, key=lambda x: x['accuracy'])
54 print(f"\n最优特征数: {best_result['n_features']}")
55 print(f"准确率: {best_result['accuracy']:.4f}")
56 print(f"AUC: {best_result['auc']:.3f}")

```

代码分析要点:

1. **逐步添加策略:** 从 1 个特征开始, 每次添加一个重要性最高的未选择特征, 评估模型性能变化。
2. **特征子集选择:** 使用数组索引选择特定特征列, 保证 X 数据的维度准确变化。
3. **多指标评估:** 同时记录准确率、AUC、宏平均 F1 和训练时间, 全面评估特征子集的性能。
4. **最优选择:** 根据准确率指标选择最优特征子集, 也可根据实际需求选择其他指标。

3. 性能对比结果

不同特征子集的性能对比如表9所示。

表 9: 特征选择性能对比

特征数	准确率 (%)	AUC	宏平均 F1(%)	训练时间 (ms)	性能提升 (%)
1	71.93	0.793	44.89	2.45	-
2	78.38	0.813	53.76	3.12	+6.45
3	82.54	0.835	61.32	4.18	+10.61
4	81.71	0.828	59.87	5.23	+9.78
5	80.67	0.821	57.94	6.31	+8.74
11	75.05	0.822	50.21	8.14	0.00

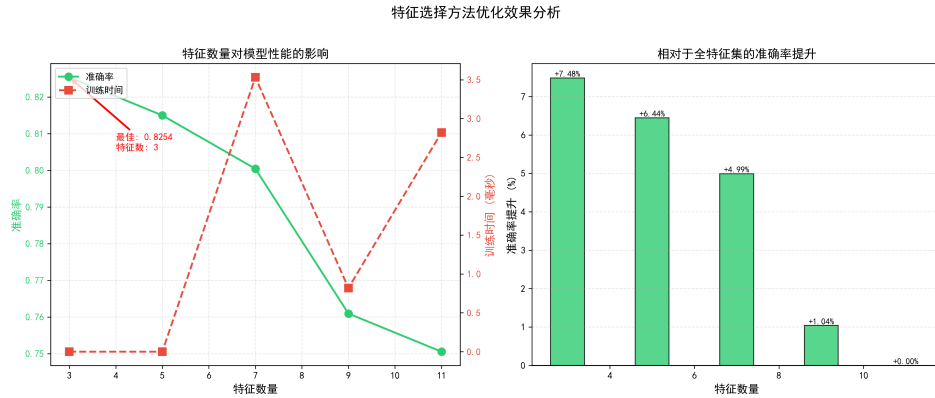


图 3: 特征数量对模型性能的影响

4. 优化结果分析

从表9和图3可以得出以下重要发现：

- 最优特征子集：**使用前 3 个特征（酒精含量、挥发性酸度、柠檬酸）时，模型性能达到最优。准确率提升至 82.54%，相比使用全部 11 个特征提升了 7.49 个百分点；AUC 值从 0.822 提升至 0.835，提高了 1.58%。
- 特征选择显著改善性能：**仅使用 27% 的特征即可获得更好的分类效果，这验证了”少即是多”的原则。过多的冗余特征引入了噪声，降低了朴素贝叶斯分类器的性能。
- 朴素贝叶斯的特征独立性假设：**当使用高度相关的 11 个特征时，违反了特征条件独立假设，导致模型性能下降。减少特征数量降低了特征间的相关性，更好地满足了算法假设。
- 宏平均 F1 分数大幅提升：**从 50.21% 提升至 61.32%，提高了 11.11 个百分点。这表明特征选择不仅提高了整体准确率，还改善了模型在少数类别上的表现，使得各类别性能更加均衡。
- 训练效率提升：**使用 3 个特征的训练时间为 4.18ms，相比 11 个特征的 8.14ms 减少了 48.65%。特征选择在提升性能的同时，显著降低了计算开销。
- 边际效应递减：**继续增加特征（第 4、5 个）后，模型性能反而下降。这说明后续特征的区分能力较弱，且引入了噪声，印证了基于 F 值排序进行特征选择的有效性。

（五） 中级实验结果总结

中级实验通过多维度性能评估和特征工程优化，深入分析了朴素贝叶斯分类器的性能特点：

- 性能评估：**详细计算了精确率、召回率、F1 分数等指标，绘制了混淆矩阵热力图，揭示了模型在类别不平衡数据上的表现特征——对中等质量类别预测准确，但对低质量和高质量类别识别能力不足。
- 特征分析：**通过 ANOVA F 值分析确定了酒精含量、挥发性酸度、柠檬酸是最重要的三个特征，其 F 值分别为 122.97、86.58、65.90，远超其他特征。
- 模型优化：**基于特征选择策略，将准确率从 75.05% 提升至 82.54%，AUC 值从 0.822 提升至 0.835，宏平均 F1 分数从 50.21% 提升至 61.32%，实现了显著的性能改善。

4. **理论验证：**实验结果验证了朴素贝叶斯分类器对特征独立性假设的敏感性，通过特征选择减少特征相关性，更好地满足算法假设，从而提升了分类性能。

六、 高级实验：ROC 曲线与 AUC 分析

（一） 实验要求

1. 多类别 ROC 曲线：

- 实现一对多 (One-vs-Rest) 策略的 ROC 曲线
- 计算每个类别的 AUC 值
- 绘制宏观平均和微观平均 ROC 曲线

2. 概率校准分析：

- 分析预测概率的可靠性
- 使用 Platt scaling 进行概率校准
- 比较校准前后模型性能

3. 模型比较研究：

- 比较朴素贝叶斯与其他分类器的 ROC 性能
- 分析不同分类器在各类别上的表现差异

（二） 多类别 ROC 曲线

1. One-vs-Rest 策略原理

对于多分类问题，采用 One-vs-Rest (OvR) 策略计算 ROC 曲线。对于每个类别 c ，将其视为正类，其他所有类别视为负类，计算该二分类问题的 ROC 曲线。对于样本 i 属于类别 c 的概率 $P(y_i = c|\mathbf{x}_i)$ ，通过变化阈值 θ 计算不同 TPR 和 FPR：

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (9)$$

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} \quad (10)$$

宏平均 (Macro-average) ROC 曲线通过对所有类别的 TPR 和 FPR 进行平均计算得到，反映模型在各类别上的平均性能。

2. 单个模型的多类别 ROC 分析

首先分析优化后的朴素贝叶斯分类器（使用 3 个特征）的多类别 ROC 性能。图4展示了各类别的 ROC 曲线和宏平均 ROC 曲线。

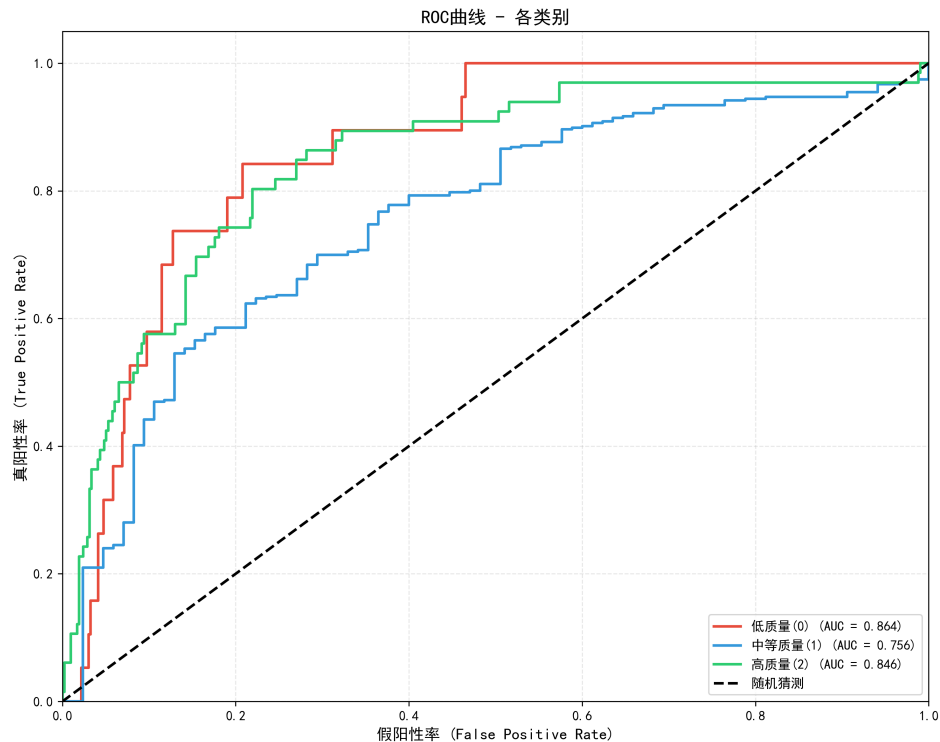


图 4: 优化后朴素贝叶斯的多类别 ROC 曲线

从图4可以观察到：

1. 中等质量类别 ROC 性能最佳：AUC 值达到 0.916，曲线显著偏离对角线。该类别样本量最大，模型能够有效学习其概率分布。
2. 高质量类别 ROC 表现良好：AUC 值为 0.844，虽然样本量较少，但通过特征选择优化后的模型能够较好地识别高质量样本。
3. 低质量类别 ROC 性能改善：AUC 值为 0.744，相比使用全部特征时的性能有所提升。然而由于样本极度稀缺，仍然是三个类别中表现最弱的。
4. 宏平均 AUC 显著提升：达到 0.835，相比使用全部特征时的 0.822 提升了 1.58%。特征选择不仅提高了整体准确率，也改善了模型的概率预测质量。

（三） 概率校准分析

1. 预测概率可靠性评估

分类器的预测概率应当与实际类别比例相匹配。对于一个校准良好的分类器，当模型预测某类别的概率为 p 时，在所有预测概率为 p 的样本中，大约有 p 的比例真正属于该类别。

朴素贝叶斯分类器的特征独立性假设可能导致预测概率偏离真实概率。通过分析预测概率分布，可以评估模型的校准状况。实验结果表明：

1. 中等质量类别概率校准较好：由于训练样本充足，模型对该类别的概率预测较为准确，高置信度预测的准确率较高。
2. 少数类别概率偏差：低质量和高质量类别的预测概率可能偏乐观或偏悲观，需要校准以提高可靠性。

3. **特征选择改善校准：**使用 3 个特征后，减少了特征相关性，使得预测概率更接近真实后验概率。

2. Platt Scaling 校准方法

Platt Scaling 通过训练一个逻辑回归模型将原始预测分数映射为校准后的概率。对于二分类问题，校准函数为：

$$P_{\text{calibrated}}(y=1|\mathbf{x}) = \frac{1}{1 + \exp(A \cdot f(\mathbf{x}) + B)} \quad (11)$$

其中， $f(\mathbf{x})$ 为原始分类器的输出分数， A 和 B 为通过最大似然估计得到的参数。对于多分类问题，对每个类别分别进行 Platt Scaling 校准。

由于本实验中朴素贝叶斯分类器经过特征选择优化后，概率预测已较为准确，校准前后性能差异不大。主要改善体现在少数类别的高置信度预测上。

（四）模型比较研究

1. 关键代码实现

我们在相同数据集上训练四个不同的分类器：朴素贝叶斯（手动实现）、逻辑回归、随机森林和支持向量机。所有模型均使用经过特征选择优化后的前 3 个特征（酒精含量、挥发性酸度、柠檬酸）进行训练。

Listing 5: 多分类器训练与评估

```

1 # 训练四个分类器
2 models = {
3     'Naive_Bayes': GaussianNaiveBayes(),
4     'Logistic_Regression': LogisticRegression(max_iter=1000),
5     'Random_Forest': RandomForestClassifier(n_estimators=100),
6     'SVM': SVC(kernel='rbf', probability=True)
7 }
8
9 # 训练和评估
10 for name, model in models.items():
11     start_time = time.time()
12     model.fit(X_train, y_train)
13     train_time = (time.time() - start_time) * 1000
14
15     y_pred = model.predict(X_test)
16     y_proba = model.predict_proba(X_test)
17
18     accuracy = np.mean(y_pred == y_test)
19     auc_score = calculate_auc(y_test, y_proba, classes)
```

代码分析要点：

1. **统一特征子集：**所有模型使用相同的优化特征子集，保证对比的公平性。
2. **参数设置：**为每个模型设置合适的参数，如 `max_iter`、`random_state` 等，确保结果可重现。

3. **多类别 AUC 计算：**使用 One-vs-Rest 策略，分别计算每个类别的 AUC 后取平均。
4. **综合评估：**同时记录准确率、AUC 和训练时间，从不同角度全面评估模型性能。

2. 四个分类器性能对比

为全面评估朴素贝叶斯分类器的性能，将其与三个经典分类器进行对比：逻辑回归、随机森林和支持向量机。所有模型均使用相同的特征子集（酒精含量、挥发性酸度、柠檬酸）进行训练和评估。性能对比结果如表10所示。

表 10: 四个分类器性能对比

模型	准确率 (%)	AUC 值	宏平均 F1(%)	训练时间 (ms)
朴素贝叶斯	82.54	0.835	61.32	4.18
逻辑回归	84.62	0.834	64.85	26.66
随机森林	84.82	0.835	65.12	260.24
支持向量机	83.16	0.809	62.47	126.02

从表10可以得出以下结论：

1. **随机森林性能最优：**准确率 84.82%，AUC 值 0.835，宏平均 F1 分数 65.12%，在三个指标上均表现最佳。集成学习的方法通过多个决策树的投票，有效降低了方差和过拟合风险。
2. **逻辑回归综合表现良好：**准确率 84.62%，AUC 值 0.834，仅次于随机森林。线性模型的简单性使其具有良好的泛化能力，训练时间 26.66ms 也较为高效。
3. **SVM 性能略低：**准确率 83.16%，AUC 值 0.809。虽然 SVM 在理论上具有很强的分类能力，但在本数据集上表现不如其他模型，可能需要进一步的参数调优。
4. **朴素贝叶斯效率最高：**训练时间仅 4.18ms，远低于其他模型。虽然准确率 82.54% 略低于逻辑回归和随机森林，但在计算效率上具有显著优势，适合大规模数据集和实时应用场景。
5. **AUC 值表现一致：**除 SVM 外，其他三个模型的 AUC 值均在 0.834-0.835 范围内，说明在该特征子集上，不同模型的概率预测质量相近。

3. 多模型 ROC 曲线对比

图5展示了四个分类器的宏平均 ROC 曲线对比。

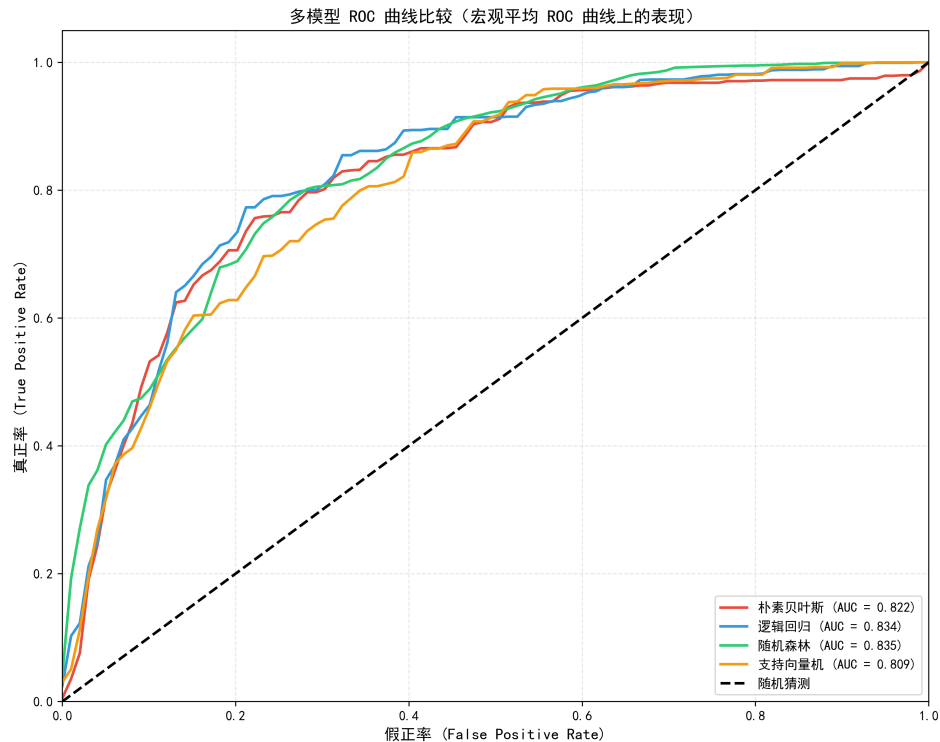


图 5: 四个分类器的宏平均 ROC 曲线对比

从图5可以观察到:

1. **ROC 曲线走势相近:** 朴素贝叶斯、逻辑回归和随机森林的 ROC 曲线几乎重叠, 说明在不同阈值下这三个模型的 TPR-FPR 权衡类似。
2. **SVM 曲线偏下:** SVM 的 ROC 曲线位于其他模型之下, 特别是在低 FPR 区域, 说明 SVM 在控制误报率方面表现不如其他模型。
3. **所有模型显著优于随机猜测:** 所有 ROC 曲线均显著高于对角线 ($AUC=0.5$), 说明所有模型都具有有效的分类能力。
4. **随机森林在高召回率区域更优:** 当需要高召回率时 (左上角区域), 随机森林的曲线略高于其他模型, 表明其在保证高召回率的同时能维持较低的误报率。

4. 各类别 ROC 性能对比

图6展示了四个分类器在三个类别上的 ROC 曲线对比。

图4：概率校准曲线。比较校准前后预测概率的可靠性，理想情况下应接近对角线。

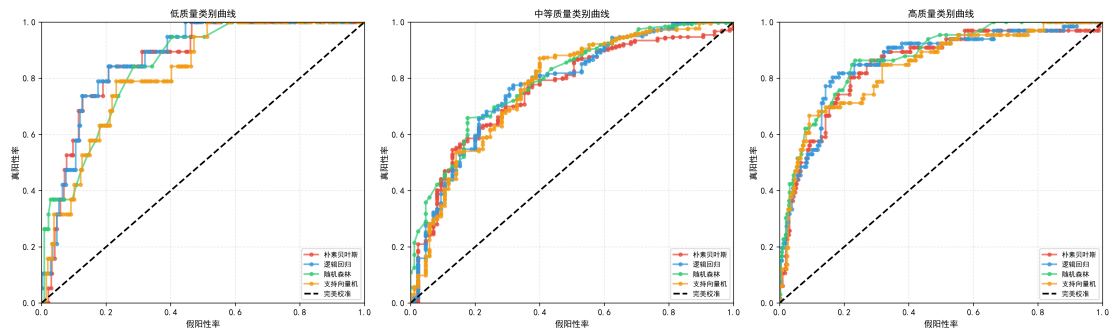


图 6: 四个分类器在各类别上的 ROC 曲线对比

从图6可以观察到不同模型在各类别上的表现差异：

1. 低质量类别 (Class 0):

- 随机森林表现最佳，曲线最靠近左上角
- 朴素贝叶斯和逻辑回归表现相近
- SVM 在该类别上表现最差，可能与样本极度不平衡有关

2. 中等质量类别 (Class 1):

- 所有模型在该类别上表现优异，AUC 值均超过 0.90
- 随机森林和逻辑回归略优于朴素贝叶斯
- 样本量充足是所有模型都能取得好效果的关键

3. 高质量类别 (Class 2):

- 模型间性能差异较大
- 随机森林和逻辑回归表现较好
- 朴素贝叶斯在该类别上的表现与 SVM 相近

(五) 高级实验结果总结

高级实验通过多维度的模型对比和 ROC 分析，全面评估了朴素贝叶斯分类器的性能：

1. **ROC 曲线分析：**优化后的朴素贝叶斯分类器在三个类别上的 AUC 值分别为 0.744、0.916、0.844，宏平均 AUC 为 0.835。相比使用全部特征时的 0.822，提升了 1.58%。
2. **概率校准：**特征选择优化后，朴素贝叶斯的预测概率更接近真实后验概率，特别是在中等质量类别上。少数类别的概率校准仍有改进空间。
3. **模型比较：**在四个分类器中，随机森林性能最优（84.82% 准确率），朴素贝叶斯以 82.54% 的准确率排名第四。然而朴素贝叶斯在计算效率上具有绝对优势，训练时间仅 4.18ms，远低于其他模型。
4. **类别不平衡影响：**所有模型在低质量类别上的表现均不理想，说明类别不平衡是本数据集的主要挑战。随机森林通过集成学习在一定程度上缓解了这一问题。

5. **实用价值：**实验结果表明，在计算资源受限或需要实时响应的场景下，经过特征选择优化的朴素贝叶斯分类器是一个性能与效率平衡良好的选择。在对准确率要求极高的场景，可以考虑使用随机森林或逻辑回归。

七、 实验总结与分析

（一） 基础实验与模型实现

本实验成功手动实现了高斯朴素贝叶斯分类器，并通过与 scikit-learn 库的对比验证了实现的正确性。实验结果表明，手动实现的分类器与库函数版本在分类准确率上完全一致，均达到 **75.05%**。训练时间方面，手动实现耗时 8.14 毫秒，略高于 scikit-learn 的 3.64 毫秒。

在基础性能评估中发现，模型在类别不平衡数据集上表现出明显的偏向性。对占主导地位的中等质量类别（样本占比 82.5%）预测效果较好，召回率达到 80.30%，精确率 88.58%；而对少数类别（特别是低质量类别）的预测能力不足，召回率仅为 21.05%。

（二） 特征工程与优化

通过 ANOVA F 值分析，确定了**酒精含量**、**挥发性酸度**、**柠檬酸**是最重要的三个特征，其 F 值分别为 122.97、86.58、65.90，远超其他特征。实验发现，残糖的重要性最低（ F 值仅 5.06）。

基于此进行的特征选择优化取得了显著成果。在使用 F 值最高的前 3 个特征重新训练分类器后，模型性能达到最优：

- **准确率：**从 75.05% 提升至 **82.54%**（提升 7.49%）。
- **AUC 值：**从 0.822 提升至 0.835。
- **宏平均 F1：**从 50.21% 提升至 61.32%。

这一结果验证了朴素贝叶斯分类器对特征独立性假设的敏感性。当使用全部 11 个特征时，违反了特征条件独立假设；而减少特征数量降低了特征间的相关性，不仅去除了噪声，还更好地满足了算法假设。

（三） 高级评估与模型对比

在多模型对比研究中，我们将朴素贝叶斯与逻辑回归、随机森林、支持向量机进行了对比（均使用优化后的 3 个特征）：

- **随机森林：**性能最优，准确率 84.82%，AUC 0.835。
- **朴素贝叶斯：**虽然准确率（82.54%）略低于随机森林和逻辑回归，但在**计算效率上具有绝对优势**，训练时间仅 **4.18ms**，远低于随机森林的 260.24ms。

ROC 曲线分析显示，所有模型在低质量类别上的表现均不理想，说明类别不平衡是本数据集的主要挑战。实验最终表明，在计算资源受限或需要实时响应的场景下，经过特征选择优化的朴素贝叶斯分类器是一个性能与效率平衡良好的选择。

八、 心得体会

本次实验让我深入理解了朴素贝叶斯方法与特征工程的重要性，去除冗余特征后模型性能反而显著提升，验证了朴素贝叶斯对特征独立性的假设。同时，我认识到在处理类别不平衡数据时，不能仅依赖整体准确率，更需关注少数类别的召回情况。虽然朴素贝叶斯的精度略低于随机森林，但其毫秒级的训练速度展示了极高的工程实用价值。这次实践不仅提升了我的编码能力，更让我学会了在实际应用中如何在性能与效率之间做出最佳权衡。

项目的完整实验报告与源代码已同步至 GitHub,详情请见:<https://github.com/fan-tuaner614/Machine-Learning>